

Projet de Génie Logiciel

Système de caisse de supermarché

Conception et implémentation en Java

Cours : Génie Logiciel

Projet : Supermarket Checkout System

Établissement : CentraleSupélec

Année universitaire : 2025–2026

Langage : Java 11+

Étudiants : Elie Saade

Elie.Saade@student-cs.fr

Alois Denoiseux

Alois.Denoiseux@student-cs.fr

Date de remise : 4 juin 2026

Table des matières

1	Introduction et objectifs	3
1.1	Démarche de conception	3
2	Analyse des exigences et traçabilité	3
3	Architecture générale	4
3.1	Rôle de chaque paquetage	5
3.2	Cas d'utilisation	5
4	Diagrammes UML	6
4.1	Diagramme de séquence — encaissement complet	9
4.2	Diagramme d'activité — achat avec livraison	9
5	Analyse détaillée des modules	10
5.1	Module <code>core</code> — la façade <code>Supermarket</code>	11
5.2	Module <code>cli</code> — l'interpréteur de commandes	11
5.3	Module <code>users</code> — acteurs et panier	12
5.4	Module <code>inventory</code> — catalogue, stock et observateur	12
5.5	Module <code>discount</code> — plans de fidélité	12
5.6	Module <code>delivery</code> — logistique et tarif dynamique	13
5.7	Module <code>payment</code> — facture, terminal et banque	13
5.8	Collections Java mobilisées	13
6	Patrons de conception	14
7	Principes de programmation orientée objet	14
7.1	Encapsulation	15
7.2	Héritage et polymorphisme	15
7.3	Programmation par interfaces	15
7.4	Principes SOLID — synthèse	15
7.5	Gestion des exceptions	15
8	Stratégie de test	15
8.1	Tests unitaires JUnit 5	15
8.2	Scénarios CLUI reproductibles	16
8.3	Comment reproduire les tests	16
9	Analyse critique : forces et limites	16
9.1	Points forts	16
9.2	Limites identifiées et pistes d'amélioration	17
10	Répartition du travail	17
11	Conclusion	18
A	Annexe — extrait du scénario <code>testScenario1.txt</code>	18

Table des figures

1	Diagramme de dépendances entre paquets. Les flèches pleines indiquent une dépendance structurelle ; les flèches en pointillés, des collaborations métier ponctuelles.	4
2	Diagramme de cas d'utilisation. Les relations « include » traduisent les étapes obligatoires de l'encaissement (autorisation bancaire, notification de stock – R9) ; la relation « extend » traduit l'option de livraison.	6
3	Diagramme de classes — cœur métier (<code>users</code> , <code>inventory</code> , <code>discount</code>). On distingue l'héritage de <code>User</code> (triangle creux), les réalisations d'interfaces <code>DiscountPolicy/CategoryPricingPolicy/S</code> (pointillés) et les compositions <code>Inventory→Category→Item</code> (losanges).	7
4	Diagramme de classes — infrastructure (<code>core</code> , <code>cli</code> , <code>payment</code> , <code>delivery</code>). La façade <code>Supermarket</code> dépend uniquement d'abstractions (<code>DiscountPolicy</code> , <code>DeliveryFeePolicy</code> , <code>DistanceCalculator</code>), ce qui matérialise le principe d'inversion des dépendances.	8
5	Diagramme de séquence du cas d'usage « encaisser un panier » (chemin nominal).	9
6	Diagramme d'activité d'un achat avec option de livraison et gestion des cas d'échec.	10

Liste des tableaux

1	Traçabilité exigences → éléments de conception.	4
3	Comportement des plans de fidélité.	12
4	Principales structures de données et justification.	13
5	Application des principes SOLID.	15
6	Description des scénarios de test fournis.	16
7	Répartition du travail.	17

1 Introduction et objectifs

Ce rapport présente la conception et l'implémentation, en Java, d'un **système de caisse de supermarché** (*Supermarket Checkout System*). Le système prend en charge le déroulement complet d'un achat : constitution d'un panier, application des politiques de prix et des plans de fidélité, calcul de la facture, encaissement sécurisé via un terminal de paiement (POS) relié à un système d'autorisation bancaire (TAS), gestion en temps réel de l'inventaire et logistique de livraison à créneaux.

L'application est pilotée par une **interface en ligne de commande** (CLUI) de type terminal Linux, conformément à la partie 2 du sujet. Le projet est accompagné d'une suite de tests **JUnit 5** couvrant chaque classe significative, d'un fichier de configuration chargé automatiquement au démarrage (`my_supermarket.ini`) et de plusieurs fichiers de scénarios reproductibles.

1.1 Démarche de conception

La conception a été guidée par deux objectifs structurants exprimés par le sujet : l'**extensibilité** (le propriétaire doit pouvoir introduire de nouveaux plans de réduction – **R5b** – ou de nouvelles politiques de prix par catégorie – **R6b**) et la **séparation des responsabilités** (caisse, banque, inventaire, livraison sont des sous-systèmes distincts). Ces deux exigences nous ont naturellement conduits vers une architecture *en couches* reposant fortement sur la **programmation par interfaces** et sur plusieurs **patrons de conception** (Stratégie, Fabrique, Observateur, Commande, Façade) décrits en section 6.

Chiffres clés du projet

- 7 paquetages, 38 classes/interfaces de production ($\approx 1\,930$ lignes).
- $\approx 1\,725$ lignes de tests, réparties en ≈ 119 méthodes `@Test` (JUnit 5).
- 1 fichier de configuration et 7 scénarios CLUI documentés.
- 5 patrons de conception mis en œuvre.

2 Analyse des exigences et traçabilité

Avant toute conception, les exigences du sujet ont été reformulées en décisions de conception concrètes. Le tableau 1 établit la **traçabilité** entre chaque exigence (R1–R10) et les éléments logiciels qui la réalisent. Cette table a servi de fil conducteur tout au long du développement et a été reprise dans la définition des scénarios de test.

Tableau 1: Traçabilité exigences → éléments de conception.

Exig.	Description résumée	Réalisation logicielle
R1	Traiter les achats des clients	Supermarket, Cart, CLI
R2	Un achat = ensemble d'articles	Cart : Map<Category, Map<Item, Integer>
R2b	Catégories fruits/légumes, laitages, viande	Category créées au setup()
R3	Articles tarifés (prix × quantité)	Item.price, CategoryPricingPolicy.apply
R4	Paiement par carte bancaire	BankCard, POSDevice, TransactionSystem
R5	Plans <i>prime</i> (20 %) / <i>platinum</i> (30 %)	PrimeDiscountPolicy, PlatinumDiscountPolicy
R5b	Ajout/modification de plans	Stratégie + Fab- rique : DiscountPolicy, DiscountPolicyFactory
R6	Politiques de prix par catégorie	PercentageCategoryPricingPolicy
R6b	Nouvelles politiques / catégories à la volée	CategoryPricingPolicy, création im- plicite de catégorie
R7	Livraison à domicile	DeliveryRequest, Customer.requestDelivery
R8	Frais selon poids et distance, refus >50 kg	WeightDistanceDeliveryFee
R8b	Remise de livraison selon le plan	DiscountPolicy.applyDeliveryDiscount
R9	Inventaire vivant + alerte seuil bas	Observateur : StockObserver, StockUnderThreshold
R10	Créneaux 2 h, capacité, tarif dynamique	DeliverySlot, DeliveryScheduler, coefficients pic/éco

3 Architecture générale

Le système est organisé en **sept paquetages** dont les dépendances suivent un flux descendant : l'interface utilisateur (**cli**) ne dialogue jamais directement avec les sous-systèmes métier ; elle passe systématiquement par la **façade** **core.Supermarket**, qui orchestre les modules **users**, **inventory**, **discount**, **delivery** et **payment**.

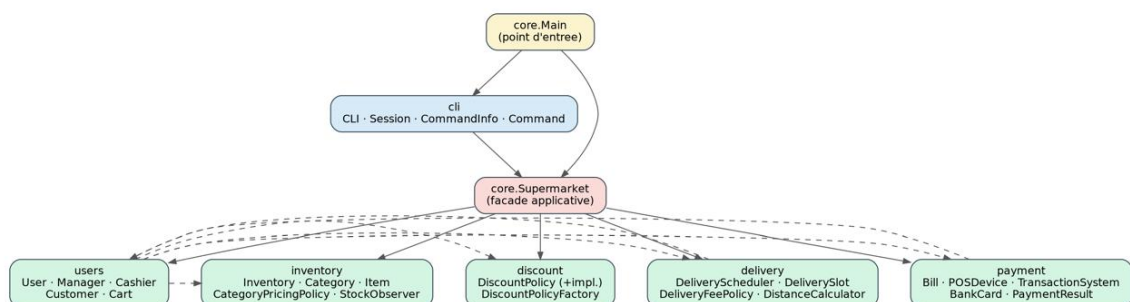


Figure 1: Diagramme de dépendances entre paquetages. Les flèches pleines indiquent une dépendance structurelle ; les flèches en pointillés, des collaborations métier ponctuelles.

3.1 Rôle de chaque paquetage

core

Point d'entrée (**Main**) et **façade** applicative (**Supermarket**) qui centralise l'état global (utilisateurs, inventaire, revenu) et coordonne les cas d'usage transverses (calcul de facture, finalisation de vente, livraison).

cli

Interpréteur de commandes : **CLI** (boucle de lecture, découpage des arguments, contrôle des droits), **Session** (état de connexion et de caisse), **CommandInfo** et l'interface fonctionnelle **Command**.

users

Hierarchie d'acteurs (**User** abstraite ; **Manager**, **Cashier**, **Customer**) et le **Cart** (panier d'achat).

inventory

Catalogue (**Inventory**, **Category**, **Item**), politiques de prix par catégorie et mécanisme d'observation du stock.

discount

Plans de fidélité (**DiscountPolicy** et ses implémentations) et leur fabrique.

delivery

Calcul des frais de livraison, créneaux horaires, capacité et estimation de distance.

payment

Facture (**Bill**), carte (**BankCard**), terminal (**POSDevice**) et système d'autorisation (**TransactionSystem**).

3.2 Cas d'utilisation

La figure 2 synthétise les acteurs et leurs interactions. Trois acteurs internes (Gestionnaire, Caissier, Client) et un acteur externe (Système bancaire / TAS) couvrent l'ensemble des commandes de la CLUI.

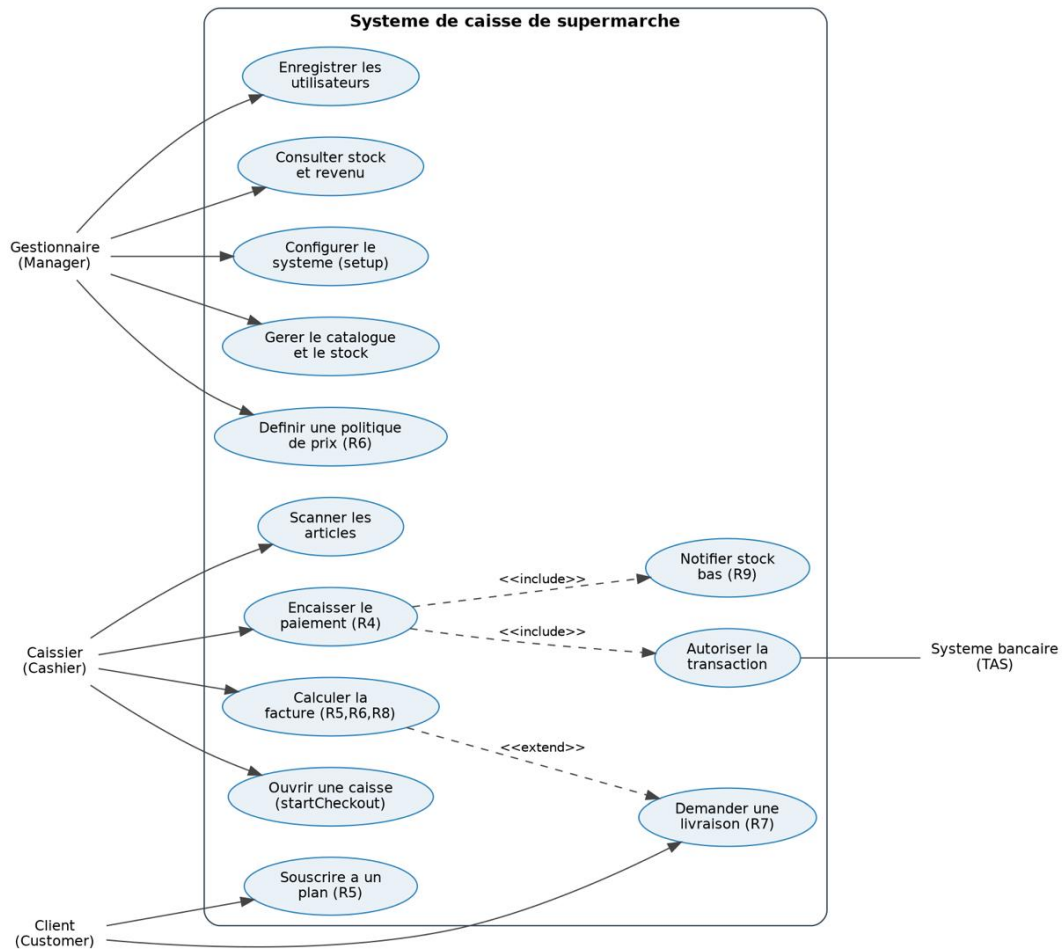


Figure 2: Diagramme de cas d'utilisation. Les relations «include» traduisent les étapes obligatoires de l'encaissement (autorisation bancaire, notification de stock – **R9**) ; la relation «extend» traduit l'option de livraison.

4 Diagrammes UML

Cette section regroupe les diagrammes structurels et comportementaux, selon les conventions usuelles de la notation UML [?]. Pour des raisons de lisibilité, le diagramme de classes est scindé en deux vues complémentaires : le *cœur métier* (figure 3) et l'*infrastructure* (figure 4).

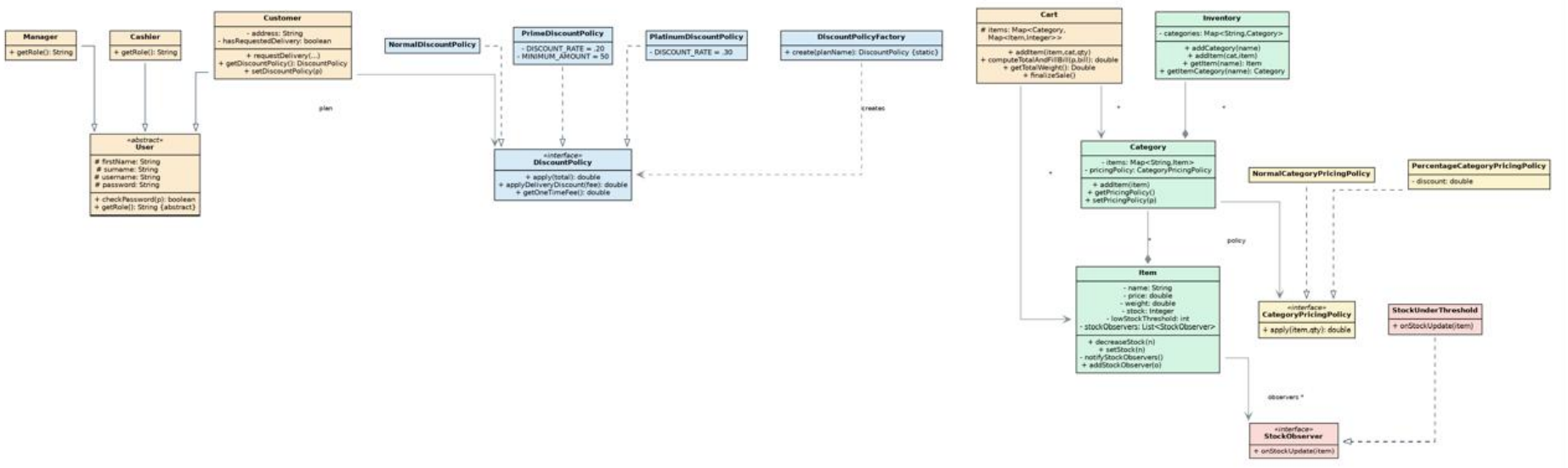


Figure 3: Diagramme de classes — cœur métier (users, inventory, discount). On distingue l'héritage de User (triangle creux), les réalisations d'interfaces DiscountPolicy/CategoryPricingPolicy/StockObserver (pointillés) et les compositions Inventory→Category→Item (losanges).

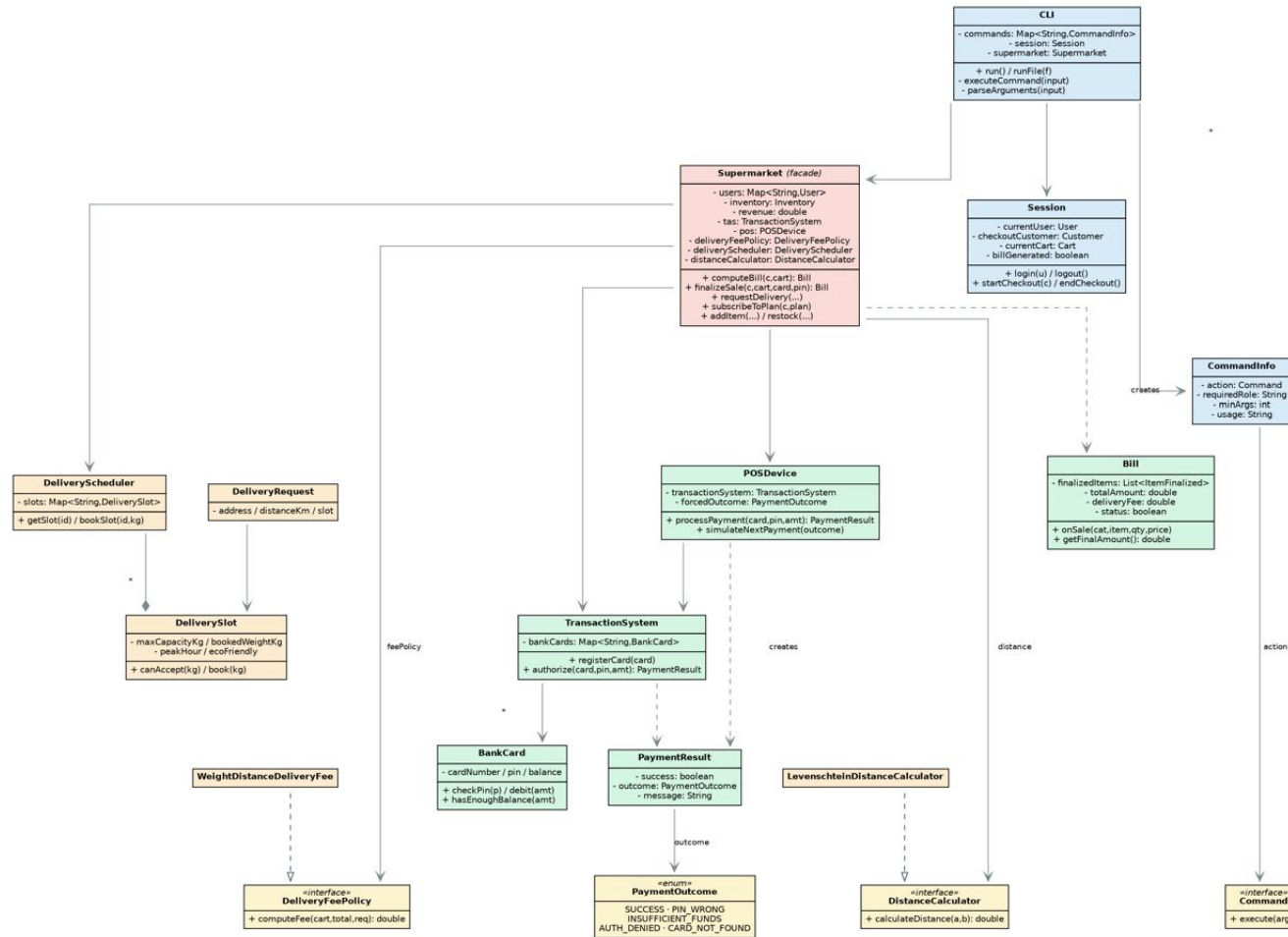


Figure 4: Diagramme de classes — infrastructure (core, cli, payment, delivery). La façade **Supermarket** dépend uniquement d'abstractions (**DiscountPolicy**, **DeliveryFeePolicy**, **DistanceCalculator**), ce qui matérialise le principe d'inversion des dépendances.

4.1 Diagramme de séquence — encaissement complet

La figure 5 décrit le scénario nominal d'un encaissement avec paiement réussi, depuis l'ouverture de la caisse jusqu'à l'impression du reçu. Il met en évidence la chaîne d'appels $CLI \rightarrow Supermarket \rightarrow Cart \rightarrow Bill$ puis $POSDevice \rightarrow TransactionSystem \rightarrow BankCard$, et la décrémentation du stock déclenchant la notification de l'observateur (**R9**).

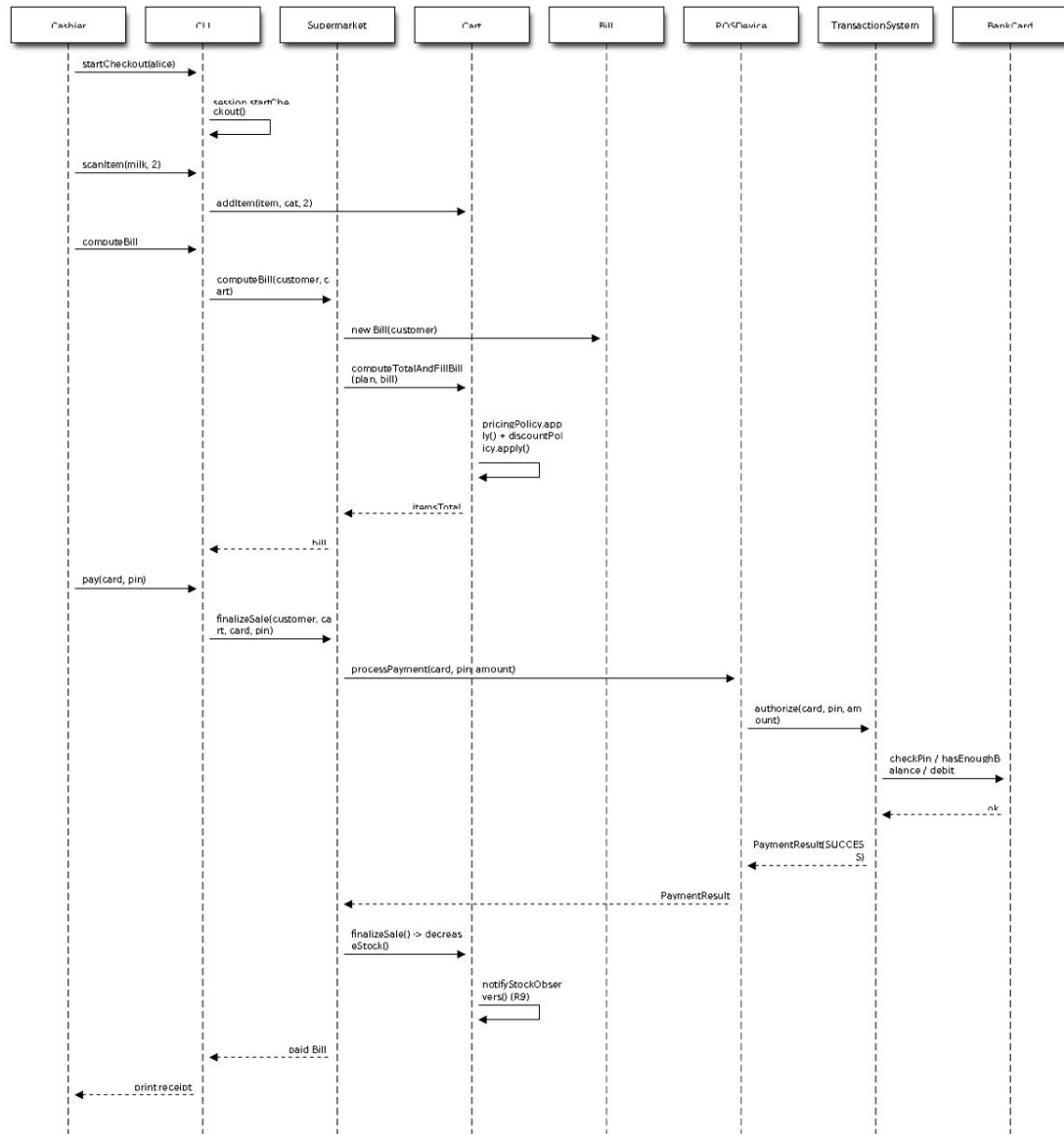


Figure 5: Diagramme de séquence du cas d'usage « encaisser un panier » (chemin nominal).

4.2 Diagramme d'activité — achat avec livraison

La figure 6 formalise le flux de contrôle complet, y compris les chemins alternatifs : stock insuffisant, livraison optionnelle, échec de paiement (le panier est conservé pour permettre une nouvelle tentative).

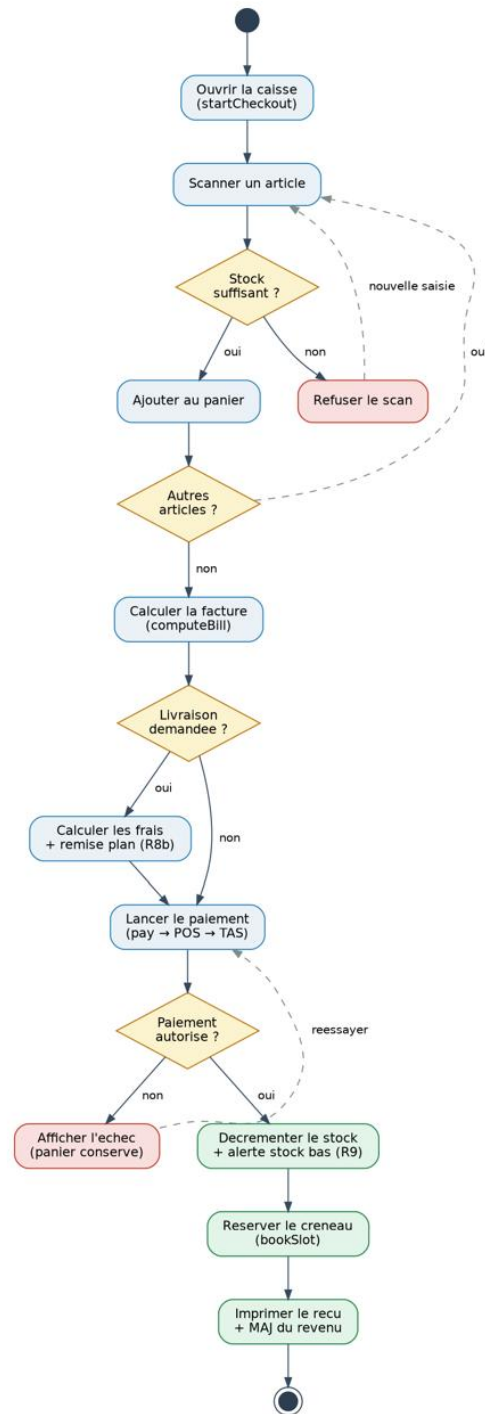


Figure 6: Diagramme d'activité d'un achat avec option de livraison et gestion des cas d'échec.

5 Analyse détaillée des modules

Nous analysons à présent chaque module en explicitant le **rôle**, les **responsabilités**, les **relations** et les **structures de données** mobilisées.

5.1 Module core — la façade Supermarket

Supermarket joue le rôle de **façade** (*Facade*) : elle expose une API métier de haut niveau (`computeBill`, `finalizeSale`, `addItem`, `requestDelivery`, `subscribeToPlan`) tout en masquant la complexité des sous-systèmes. Elle conserve l'état global dans des collections : `Map<String,User>` indexée par nom d'utilisateur, un `Inventory`, le revenu cumulé, ainsi que les instances de `POSDevice`, `TransactionSystem`, `DeliveryScheduler` et des stratégies de calcul.

Le point le plus intéressant est la méthode `finalizeSale` : elle illustre une **transaction métier** qui n'engage les effets de bord (débit, décrétement de stock, réservation de créneau, mise à jour du revenu) qu'*après* confirmation du paiement.

```

1 public Bill finalizeSale(Customer customer, Cart cart, String cardNumber, String pin) {
2     Bill bill = computeBill(customer, cart);
3     PaymentResult result = getPosDevice().processPayment(cardNumber, pin, bill.getFinalAmount())
4     ;
5     if (!result.isSuccess()) {           // chemin d'échec : aucun effet de bord
6         System.out.println(result.getMessage());
7         return null;
8     }
9     if (customer.hasRequestedDelivery()) {
10        DeliveryRequest request = customer.getDeliveryRequest();
11        deliveryScheduler.bookSlot(request.getSlot().getSlotId(), cart.getTotalWeight());
12    }
13    bill.setStatus(true);
14    addRevenue(bill.getFinalAmount());
15    cart.finalizeSale();                 // decremente le stock -> declenche R9
16    customer.clearDeliveryRequest();
17    return bill;
18 }

```

Extrait 1: `Supermarket.finalizeSale` — finalisation conditionnée au succès du paiement.

5.2 Module cli — l'interpréteur de commandes

La classe CLI maintient un registre `Map<String,CommandInfo>`. Chaque `CommandInfo` agrège : une description, le **rôle requis** (contrôle d'accès), le nombre minimal d'arguments, un message d'usage et une **action** typée par l'interface fonctionnelle `Command`. Les actions sont enregistrées sous forme de **références de méthodes** (`this::login`, `this::pay`, etc.), ce qui constitue une mise en œuvre élégante du **patron Commande** adossée aux *lambdas* de Java 8+.

```

1 commands.put("pay", new CommandInfo("Pay the bill", this::pay,
2     "Cashier", 2, "pay <cardNumber> <pin>"));
3 commands.put("scanitem", new CommandInfo("Scan an item", this::scanItem,
4     "Cashier", 2, "scanitem <item_id> <quantity>"));

```

Extrait 2: Enregistrement déclaratif des commandes (patron Commande).

La boucle `executeCommand` applique, dans l'ordre : analyse lexicale (`parseArguments`, qui gère les guillemets pour les arguments contenant des espaces, ex. une adresse), contrôle de permission (`checkPermission`), contrôle arité (`checkMinArgsLength`), puis exécution encapsulée dans un `try/catch` qui transforme toute exception métier en message lisible. Ce dispositif répond directement à l'exigence du sujet sur la **gestion exhaustive des erreurs**.

La classe `Session` encapsule la machine à états de l'interaction : utilisateur connecté, client et panier de la caisse en cours, et un drapeau `billGenerated`. Ce dernier impose un **invariant** important : toute modification du panier (`scanItem`, `setCategoryDiscount`, `requestDelivery`) repositionne `billGenerated=false`, forçant un recalcul (`computeBill`) avant tout paiement. On évite ainsi de payer un montant périmé.

5.3 Module users — acteurs et panier

La classe abstraite `User` factorise les attributs communs et la vérification de mot de passe ; la méthode abstraite `getRole()` est **polymorphe** et sert à la fois à l’affichage et au contrôle d’accès. `Customer` enrichit l’acteur d’un `DiscountPolicy` (composition) et d’un état de livraison.

Le `Cart` repose sur une structure imbriquée `Map<Category, Map<Item, Integer>`. Ce choix regroupe les articles par catégorie, ce qui est précieux car la **politique de prix s’applique au niveau de la catégorie** : on peut donc parcourir chaque catégorie et appliquer directement sa `pricingPolicy`. La méthode `computeTotalAndFillBill` cumule le total, peuple la facture ligne à ligne, puis applique le plan client au total agrégé.

```

1 double total = 0;
2 for (Map.Entry<Category, Map<Item, Integer>> categoryEntry : items.entrySet()) {
3     Category category = categoryEntry.getKey();
4     for (Map.Entry<Item, Integer> itemEntry : categoryEntry.getValue().entrySet()) {
5         Item item = itemEntry.getKey();
6         int quantity = itemEntry.getValue();
7         double linePrice = category.getPricingPolicy().apply(item, quantity); // R6
8         total += linePrice;
9         bill.onSale(category, item, quantity, linePrice);
10    }
11 }
12 total = discountPolicy.apply(total); // R5 : plan client sur le total
13 bill.setTotalAmount(total);

```

Extrait 3: `Cart.computeTotalAndFillBill` — agrégation par catégorie puis plan client.

5.4 Module inventory — catalogue, stock et observateur

`Inventory` agrège des `Category`, qui agrègent des `Item`. La recherche d’un article (`getItem`) parcourt les catégories : un compromis assumé qui privilégie la simplicité, le catalogue restant de taille modeste.

`Item` est le **sujet** d’un patron `Observateur` : il détient une liste d’`StockObserver` et appelle `notifyStockObservers()` à chaque modification de stock (`setStock`, `decreaseStock`). L’implémentation `StockUnderThreshold` émet une alerte lorsque le stock passe sous un seuil (**R9**). Le constructeur de `Item` valide ses invariants (nom non vide, prix/poids/stock positifs) et lève des `IllegalArgumentException` explicites.

`CategoryPricingPolicy` est une **interface de stratégie** : `NormalCategoryPricingPolicy` (prix plein) ou `PercentageCategoryPricingPolicy` (réduction en pourcentage). Le gestionnaire peut donc, via `setCategoryDiscount`, remplacer dynamiquement la politique d’une catégorie – ce qui répond à **R6b**.

5.5 Module discount — plans de fidélité

L’interface `DiscountPolicy` expose trois opérations : `apply(total)` (remise sur le panier), `applyDeliveryDiscount` (remise sur la livraison, **R8b**) et `getOneTimeFee()` (cotisation annuelle). Les trois implémentations encodent les règles métier :

Tableau 3: Comportement des plans de fidélité.

Plan	Remise panier	Condition	Remise livraison
normal	0 %	–	0 %
prime	20 %	total ≥ 50 €	50 %
platinum	30 %	toujours	100 % (gratuite)

`DiscountPolicyFactory.create(planName)` centralise l'instanciation (**Fabrique simple**). L'ajout d'un nouveau plan se limite à créer une classe implémentant `DiscountPolicy` et à l'enregistrer dans la fabrique – le reste du système (façade, panier, facture) reste inchangé. C'est exactement la flexibilité exigée par **R5b**.

5.6 Module *delivery* — logistique et tarif dynamique

Ce module met en œuvre l'extension avancée **R10**. `DeliveryScheduler` initialise quatre créneaux de 2 heures (*morning, lunch, afternoon, evening*) caractérisés par une **capacité** (100 kg), un indicateur d'**heure de pointe** et un indicateur **éco-responsable**. `DeliverySlot.canAccept / book` empêchent le **surbooking**.

`WeightDistanceDeliveryFee` implémente l'interface `DeliveryFeePolicy` : refus au-delà de 50 kg, forfait de 15 € pour les petites livraisons proches, sinon forfait + 5 % du total, modulé par un coefficient de pointe (1,3) ou éco-responsable (0,9). Enfin, `DistanceCalculator` est une interface dont l'implémentation fournie (`LevenshteinDistanceCalculator`) *simule* la distance géographique par une distance d'édition normalisée entre adresses – choix discuté en section 9.

5.7 Module *payment* — facture, terminal et banque

La chaîne de paiement sépare clairement les responsabilités :

- `Bill` agrège les lignes finalisées (classe interne `ItemFinalized`), le sous-total, les frais de livraison et l'état *payé/impayé*.
- `POSDevice` joue le rôle d'intermédiaire (terminal) et possède un **point d'injection de test** `simulateNextPayment` permettant de forcer l'issue du prochain paiement – indispensable pour reproduire les scénarios « malheureux » sans manipuler l'état bancaire.
- `TransactionSystem` (TAS) détient les cartes et réalise la séquence d'autorisation : carte connue ? PIN correct ? solde suffisant ? puis débit.
- `PaymentResult` et l'énumération `PaymentOutcome` constituent un **objet-résultat** : plutôt que de propager des exceptions pour des cas d'échec attendus (PIN erroné, fonds insuffisants), le système renvoie un résultat explicite et typé.

5.8 Collections Java mobilisées

Tableau 4: Principales structures de données et justification.

Structure	Emplacement	Justification
<code>HashMap<String, User></code>	<code>Supermarket.users</code>	accès $O(1)$ par identifiant
<code>HashMap<String, Category></code>	<code>Inventory</code>	recherche de catégorie par nom
<code>HashMap<String, Item></code>	<code>Category.items</code>	recherche d'article par nom
<code>Map<Category, Map<Item, Integer> items</code>		regroupement par catégorie pour appliquer R6
<code>HashMap<String, BankCard></code>	<code>TransactionSystem</code>	autorisation par numéro de carte
<code>HashMap<String, DeliverySlot></code>	<code>DeliveryScheduler</code>	accès direct au créneau
<code>ArrayList<StockObserver></code>	<code>Item</code>	itération sur les observateurs
<code>ArrayList<ItemFinalized></code>	<code>Bill</code>	lignes ordonnées de la facture

6 Patrons de conception

Cinq patrons structurent la solution, au sens du catalogue classique du *Gang of Four* [?]. Pour chacun, nous précisons le **problème résolu**, la **mise en œuvre** et le **bénéfice**.

Stratégie (Strategy)

Problème. **R5b/R6b/R8** exigent de faire varier des *algorithmes* (remise client, prix par catégorie, frais de livraison, calcul de distance) sans modifier les classes clientes.

Mise en œuvre. Quatre familles de stratégies : `DiscountPolicy`, `CategoryPricingPolicy`, `DeliveryFeePolicy` et `DistanceCalculator`. La façade et le panier ne manipulent que ces interfaces.

Bénéfice. Respect du **principe ouvert/fermé** : une nouvelle règle = une nouvelle classe, sans toucher à l'existant.

Fabrique (Factory Method / Simple Factory)

Problème. Instancier le bon plan à partir d'une chaîne ("prime", "platinum"...) saisie en CLUI.

Mise en œuvre. `DiscountPolicyFactory.create` centralise le `switch` sur le nom de plan et lève une exception sur un plan inconnu.

Bénéfice. Une **unique** source de vérité pour la création des plans ; le couplage des appelants aux classes concrètes est éliminé.

Observateur (Observer) — exigence **R9**

Problème. Notifier un module de gestion lorsque le stock d'un article passe sous un seuil.

Mise en œuvre. `Item` (sujet) notifie ses `StockObserver` à chaque variation de stock ; `StockUnderThreshold` (observateur concret) émet l'alerte.

Bénéfice. Découplage total entre la logique de stock et la logique d'alerte ; on peut brancher d'autres observateurs (réapprovisionnement automatique, journal...).

Commande (Command)

Problème. Associer dynamiquement un nom de commande textuelle à un traitement, tout en gérant droits et arité de façon uniforme.

Mise en œuvre. Interface fonctionnelle `Command`, descripteurs `CommandInfo`, et un registre `Map<String,CommandInfo>` peuplé de références de méthodes.

Bénéfice. Ajouter une commande = une ligne dans `registerCommands` ; la boucle d'exécution reste générique.

Façade (Facade)

Problème. Offrir à la CLUI un point d'entrée unique et cohérent vers des sous-systèmes hétérogènes.

Mise en œuvre. `Supermarket` coordonne `users`, `inventory`, `discount`, `delivery`, `payment`.

Bénéfice. L'interface ne connaît qu'une seule classe métier ; les sous-systèmes peuvent évoluer indépendamment.

7 Principes de programmation orientée objet

7.1 Encapsulation

Les attributs sont systématiquement `private` (ou `protected` pour les champs hérités de `User`) et exposés via des accesseurs, conformément aux bonnes pratiques de conception d'API en Java [?]. Les **invariants** sont protégés dès la construction : `Item`, `BankCard` et `PercentageCategoryPricingPolicy` rejettent les valeurs invalides dans leur constructeur.

7.2 Héritage et polymorphisme

`Manager`, `Cashier` et `Customer` héritent de `User` et redéfinissent `getRole()`. Le contrôle d'accès de la CLUI exploite ce **polymorphisme** : `checkPermission` compare simplement `user.getRole()` au rôle requis, sans connaître le sous-type concret. La substituabilité (**LSP**) est respectée : tout `User` peut être traité uniformément.

7.3 Programmation par interfaces

Sept interfaces (`Command`, `StockObserver`, `CategoryPricingPolicy`, `DiscountPolicy`, `DeliveryFeePolicy`, `DistanceCalculator`) découplent les contrats des implémentations. Elles sont volontairement **pe-tites** (principe de ségrégation des interfaces, **ISP**).

7.4 Principes SOLID — synthèse

Tableau 5: Application des principes SOLID.

Pr.	Illustration dans le projet
S	Responsabilité unique : <code>TransactionSystem</code> autorise, <code>POSDevice</code> relaie, <code>Bill</code> agrège, <code>Cart</code> accumule.
O	Ouvert/fermé : ajout de plans/politiques par de nouvelles classes (stratégie + fabrique).
L	Substitution de Liskov : les sous-types de <code>User</code> et les stratégies sont interchangeables.
I	Ségrégation : interfaces minimales et ciblées.
D	Inversion des dépendances : <code>Supermarket</code> et <code>POSDevice</code> dépendent d'abstractions, pas de classes concrètes.

7.5 Gestion des exceptions

Le projet distingue deux régimes. Les **violations d'invariants** / **mauvais usages** lèvent des exceptions non vérifiées (`IllegalArgumentException`, `IllegalStateException`) interceptées et formatées par la CLUI. Les **échecs métier attendus** du paiement (PIN erroné, fonds insuffisants, carte inconnue) ne sont *pas* traités par exception mais via l'objet-résultat `PaymentResult`/`PaymentOutcome`, distinction de conception qui clarifie le code appelant.

8 Stratégie de test

8.1 Tests unitaires JUnit 5

Chaque classe significative possède sa classe de test miroir (par exemple `TransactionSystemTest`, `WeightDistanceDeliveryFeeTest`, `CartTest`, `ItemTest`). Au total, le projet comporte ≈ 119 méthodes `@Test` réparties sur 34 classes de test, couvrant notamment : le calcul des remises selon le seuil de 50 € (prime), le refus de livraison au-delà de 50 kg, les coefficients de tarif dynamique, la décrémentation de stock et le déclenchement de l'observateur, et les quatre issues de paiement.

8.2 Scénarios CLUI reproductibles

Le système charge automatiquement `my_supermarket.ini` au démarrage (équivalent du `setup` enrichi) puis exécute des scénarios via `runTest <fichier>`. Le tableau 6 décrit chaque scénario fourni.

Tableau 6: Description des scénarios de test fournis.

Fichier	Objet du test
<code>testScenario1.txt</code>	Scénario de bout en bout : catalogue, remise catégorie (R6), plan <i>prime</i> (R5), panier mixte (R2b), livraison (R7,R8,R8b), échec puis succès de paiement, alerte de stock bas (R9).
<code>test_basic_checkout.txt</code>	Encaissement nominal sans livraison ; décrétement de stock et mise à jour du revenu.
<code>test_discounts_and_plans.txt</code>	Politiques de prix par catégorie et plans <i>normal/prime/platinum</i> (seuil de remise).
<code>test_payment_system.txt</code>	Issues de paiement forcées (<code>PIN_WRONG</code> , <code>INSUFFICIENT_FUNDS</code> , <code>AUTH_DENIED</code>), carte incon- nue, puis succès.
<code>test_inventory_stock.txt</code>	Décrétement après paiement, alerte de stock bas, réappro- visionnement.
<code>test_delivery_slots_pricing.txt</code>	Créneaux et tarif dynamique : pointe, éco-responsable, remises <i>prime/platinum</i> sur la livraison.
<code>test_error_handling_permissions.txt</code>	Scénario négatif : commandes sans connexion, mauvais mot de passe, erreurs de permission, articles/clients in- connus.

8.3 Comment reproduire les tests

Procédure de test

Tests unitaires (Maven) :

```
mvn test
```

Lancer l'application (charge `my_supermarket.ini` au démarrage) :

```
mvn exec:java
# ou, sans Maven :
mkdir -p out && javac -d out $(find src/main/java -name "*.java")
java -cp out core.Main
```

Rejouer un scénario depuis l'invite de la CLUI :

```
> runTest testScenario1.txt
> runfile scenarios/test_delivery_slots_pricing.txt
```

9 Analyse critique : forces et limites

9.1 Points forts

- **Extensibilité réelle** : l'association Stratégie + Fabrique satisfait pleinement **R5b** et **R6b**, vérifiée par les scénarios.

- **Séparation des préoccupations** nette entre interface, façade et métier ; la CLUI est purement « présentation ».
- **Testabilité** : `simulateNextPayment` et l'injection de `DistanceCalculator` / `DeliveryFeePolicy` créent des coutures de test propres.
- **Gestion d'erreurs robuste** : la boucle CLUI ne se bloque jamais sur une saisie erronée.

9.2 Limites identifiées et pistes d'amélioration

Une lecture critique du code révèle plusieurs points perfectibles, que nous assumons et documentons :

- **Distance simulée** : `LevenshteinDistanceCalculator` approxime la distance géographique par une distance d'édition entre adresses. C'est un *placeholder* pédagogique ; l'interface `DistanceCalculator` permet toutefois de brancher un véritable service de géocodage sans rien changer d'autre.
- **Cotisation de plan** : `subscribeToPlan` ajoute la cotisation au revenu sans la débiter d'une carte (commentaire explicite dans le code).
- **Recherche d'article** en $O(\text{nombre de catégories})$: acceptable ici, mais un index global `Map<String,Item>` serait préférable à grande échelle.

10 Répartition du travail

Le projet a été mené en **binôme** par Elie Saade et Alois Denoiseux. Plutôt qu'un découpage en lots attribués individuellement, nous avons travaillé de manière **collaborative sur l'ensemble des axes** du projet : la conception (architecture, choix des patrons), l'implémentation des sept paquetages, la réalisation des diagrammes UML et l'écriture des tests JUnit ont fait l'objet d'un effort partagé, par échanges réguliers et relectures croisées. Le tableau 7 synthétise cette contribution commune selon les axes imposés par le sujet.

Tableau 7: Répartition du travail.

Axe	Contribution	Périmètre
Design / architecture	Commune	Architecture en couches, façade, choix et intégration des patrons (Stratégie, Fabrique, Observateur, Commande).
Code	Commune	Sept paquetages <code>core</code> , <code>cli</code> , <code>users</code> , <code>inventory</code> , <code>discount</code> , <code>delivery</code> , <code>payment</code> .
Diagrammes UML	Commune	Classes (domaine, infrastructure), cas d'utilisation, séquence, activité, architecture.
Tests JUnit	Commune	Tests unitaires de chaque classe significative et scénarios CLUI reproductibles.

L'ensemble du travail (conception, code, diagrammes et tests) a été réalisé conjointement par les deux membres du binôme.

11 Conclusion

Le système réalisé couvre l'intégralité des exigences fonctionnelles, des fonctions de base (R1–R8) aux extensions avancées de gestion d'inventaire (**R9**) et de logistique dynamique (**R10**). Au-delà de la conformité, l'effort a porté sur la **qualité de conception** : une architecture en couches, une façade claire et un recours assumé aux patrons de conception rendent le système **extensible** et **testable**. L'analyse critique met en lumière des pistes concrètes (distance réelle, configuration du seuil de stock, cohérence du tarif forfaitaire) qui constitueraient les prochaines itérations naturelles du projet.

A Annexe — extrait du scénario testScenario1.txt

```
login ceo 123456789
setup
addItem milk dairy 1.20 1.00 50
setCategoryDiscount fruit-and-vegetables 10
registerCustomer Alice Martin alice "12 rue de la Paix" alicepwd
...
login alice alicepwd
subscribeToPlan prime
requestDelivery "12 rue de la Paix"
logout
login bob bobpwd
startCheckout alice
scanItem milk 2
computeBill
simulatePayment INSUFFICIENT_FUNDS
pay 4242424242424242 1234
simulatePayment SUCCESS
pay 4242424242424242 1234
```